LAPORAN PRAKTIKUM

JOBSHEET 7 - PEMROGRAMAN PHP DASAR DAN FORM HANDLING



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Modularisasi Antarmuka dengan Header dan Footer 3

2.1.1. 1.1 Penerapan require_once dan konstanta `__DIR__` 3

2.1.2. Langkah-langkah 4

2.1.3. Pertanyaan 5

2.2. Percobaan 2: Pemrosesan Form Tambah Data dan Validasi Sisi Server 6

2.2.1. 2.1 Sanitasi Input dan Validasi Berlapis 6

2.2.2. 2.2 Penolakan Data Tidak Valid 7

2.2.3. Langkah-langkah 7

2.2.4. Pertanyaan 8

2.3. Percobaan 3: Penyimpanan Data pada Session dan Arsitektur PRG 9

2.3.1. 3.1 Pola Navigasi Post/Redirect/Get (PRG) 9

2.3.2. 3.2 Notifikasi Flash Sekali-Pakai (Flash Message) 9

2.3.3. Langkah-langkah 10

2.3.4. Pertanyaan 12

3. Tugas 13

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 7 13

1. Tujuan

- Memahami konsep dasar arsitektur Server-Side Rendering (SSR) menggunakan bahasa pemrograman PHP.
- Menguasai teknik modularisasi antarmuka web mematuhi prinsip DRY (Don't Repeat Yourself) melalui direktif require_once dan konstanta absolut `__DIR__`.
- Menerapkan penanganan status (state management) pada protokol stateless HTTP memanfaatkan variabel superglobal `$_SESSION`.
- Mengimplementasikan pemrosesan data form masukan melalui metode HTTP POST dan sanitasi nilai masukan.
- Membangun lapisan validasi data sisi server (Server-Side Validation) yang tangguh dan tidak dapat dimanipulasi oleh peramban klien.
- Menerapkan arsitektur navigasi Post/Redirect/Get (PRG) menggunakan fungsi header('Location: ...') dan terminasi eksekusi exit.
- Merancang sistem notifikasi sekali-pakai (Flash Messages) berbasis session dengan mekanisme pembersihan variabel unset().
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/DnPWeb2026.

2. Praktikum

2.1. Percobaan 1: Modularisasi Antarmuka dengan Header dan Footer

Pada percobaan pertama, struktur dokumen aplikasi SIMPUS-Mini dimigrasikan dari file statis HTML ke skrip PHP dinamis. Kerangka tata letak yang berulang seperti navigasi `<header>` dan penutup `<footer>` dipisahkan ke dalam folder `includes/`.

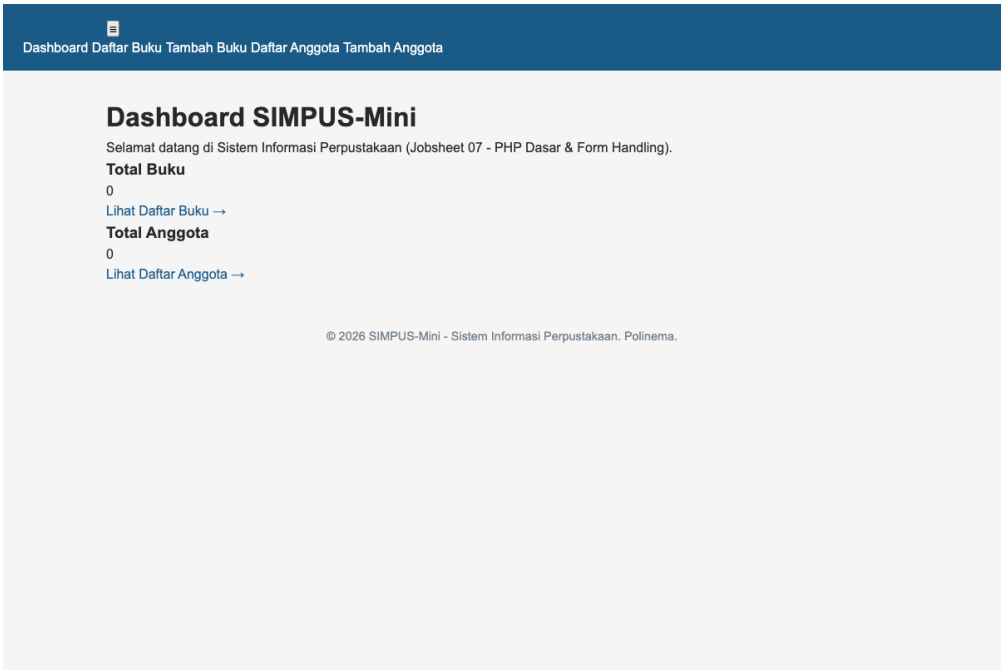
2.1.1. 1.1 Penerapan require_once dan konstanta `__DIR__`

Modularisasi dilakukan menggunakan fungsi bawaan `require_once` yang dipadukan dengan konstanta direktori absolut `__DIR__`:

```
<?php
$page_title = "Dashboard";
require_once __DIR__ . '/includes/header.php';
?>
```

Penggunaan `__DIR__` menjamin keakuratan resolusi path berkas di lingkungan server tanpa terpengaruh oleh direktori kerja peramban. Di dalam `header.php`,

sesi diinisialisasi secara terpusat melalui `session_start()`, dan komponen navigasi utama disajikan seragam untuk seluruh halaman.



Gambar 1.1: Antarmuka Dashboard SIMPUS-Mini Menampilkan Statistik Data Buku dan Anggota dari Sesi.

2.1.2. Langkah-langkah

1. Membuat berkas `includes/header.php` yang memuat pembukaan tag HTML, inisialisasi session_start(), serta elemen header navbar.
2. Membuat berkas `includes/footer.php` yang memuat penutup tag dokumen HTML.
3. Mengintegrasikan kedua berkas pembungkus pada `index.php` menggunakan direktif `require_once`.
4. Memverifikasi tata letak antarmuka di peramban web pada alamat `http://localhost:8000/jobsheet07/src/index.php`.

2.1.3. Pertanyaan

1. Mengapa penggunaan `require_once` lebih aman dibandingkan `include` pada pemuatan berkas inti?
    - `require_once` menghasilkan kesalahan fatal (**Fatal Error: E_COMPILE_ERROR**) dan menghentikan seluruh eksekusi program jika berkas yang ditargetkan tidak ditemukan, mencegah berjalannya skrip dalam kondisi dependensi yang rusak. Selain itu, imbuhan `_once` memastikan berkas tersebut hanya dimuat tepat satu kali, mencegah terjadinya deklarasi ulang fungsi atau variabel ganda (**Cannot redeclare function**).

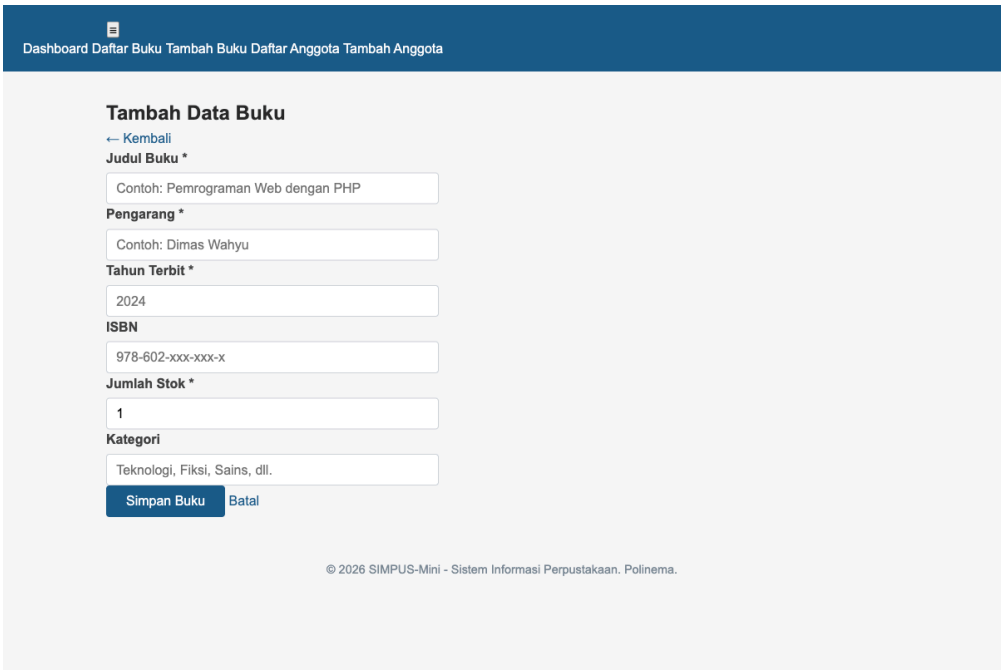
2.2. Percobaan 2: Pemrosesan Form Tambah Data dan Validasi Sisi Server

Percobaan kedua mengimplementasikan alur penerimaan data formulir tambah buku dan anggota melalui metode HTTP POST pada skrip pengendali `proses_tambah.php`.

2.2.1. 2.1 Sanitasi Input dan Validasi Berlapis

Data masukan pengguna diekstrak dari variabel superglobal `$_POST` dan disanitasi menggunakan fungsi `trim()`. Berbeda dengan Jobsheet 5 yang mengandalkan JavaScript di peramban, validasi di Jobsheet 7 dieksekusi murni di sisi server:

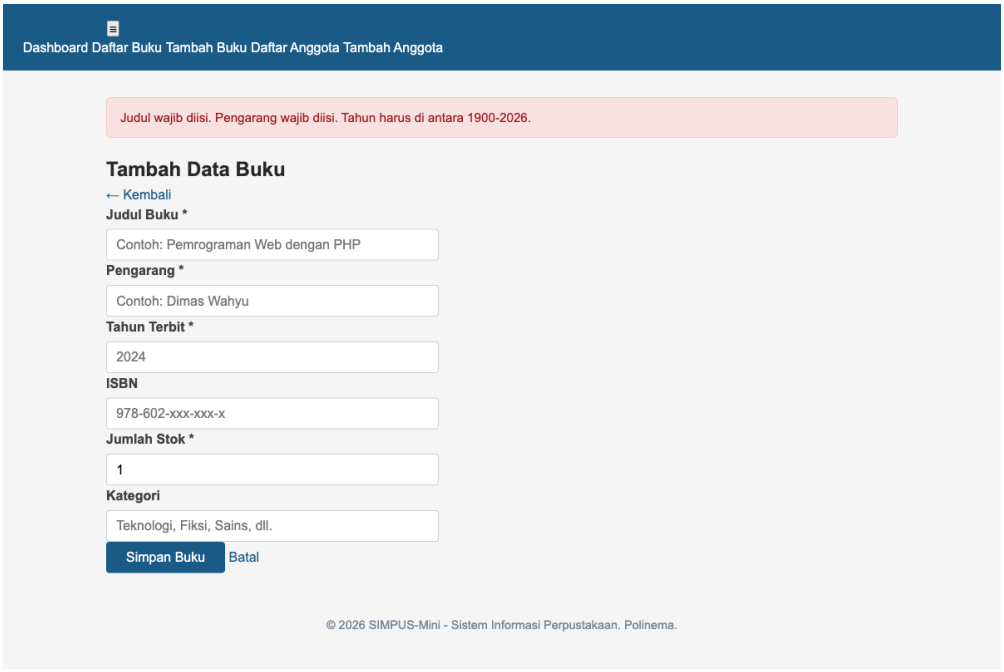
```
$errors = [];
if ($judul === '') {
    $errors[] = "Judul wajib diisi.";
}
if ($pengarang === '') {
    $errors[] = "Pengarang wajib diisi.";
}
if (!is_numeric($tahun) || $tahun < 1900 || $tahun > 2026) {
    $errors[] = "Tahun harus di antara 1900-2026.";
}
if (!is_numeric($stok) || $stok < 0) {
    $errors[] = "Stok tidak boleh negatif.";
}
```



Gambar 1.2: Tampilan Form Tambah Data Buku dengan Berbagai Bidang Isian Terstruktur.

2.2.2. 2.2 Penolakan Data Tidak Valid

Apabila array `$errors` tidak kosong, alur eksekusi langsung membatalkan penyimpanan data, mencatat pesan kesalahan ke dalam variabel flash session, dan membelokkan kembali pengguna ke form input:



Gambar 1.3: Notifikasi Galat Server-Side Berwarna Merah Muncul Saat Bidang Form Wajib Dikosongkan.

2.2.3. Langkah-langkah

1. Menyusun form HTML pada `books/tambah.php` dengan atribut `action="proses_tambah.php" method="POST"`.
2. Mengembangkan skrip pemrosesan `books/proses_tambah.php` yang mengekstrak nilai `$_POST`.
3. Menulis aturan validasi kelayakan data (keberadaan teks, rentang tahun, dan batas minimal stok).
4. Menguji pengiriman formulir dengan data tidak valid dan memverifikasi penolakan penyimpanan di sisi server.

2.2.4. Pertanyaan

1. Mengapa validasi sisi server mutlak diperlukan meskipun validasi client-side sudah ada?

    - Validasi sisi klien dapat dilewati dengan sangat mudah oleh pengguna melalui penonaktifan JavaScript di peramban atau pengiriman request langsung menggunakan cURL dan Postman. Validasi sisi server merupakan satu-satunya lapisan pertahanan yang berada 100% di bawah kendali pengembang dan tidak dapat dimanipulasi oleh klien.

2.3. Percobaan 3: Penyimpanan Data pada Session dan Arsitektur PRG

Percobaan ketiga mengimplementasikan persistensi data sementara pada variabel superglobal `$_SESSION['buku']` dan menerapkan pola navigasi Post/Redirect/Get (PRG).

2.3.1. 3.1 Pola Navigasi Post/Redirect/Get (PRG)

Setelah data buku yang valid ditambahkan ke array sesi, server tidak langsung merender halaman HTML, melainkan mengirimkan header pengalihan lokasi:

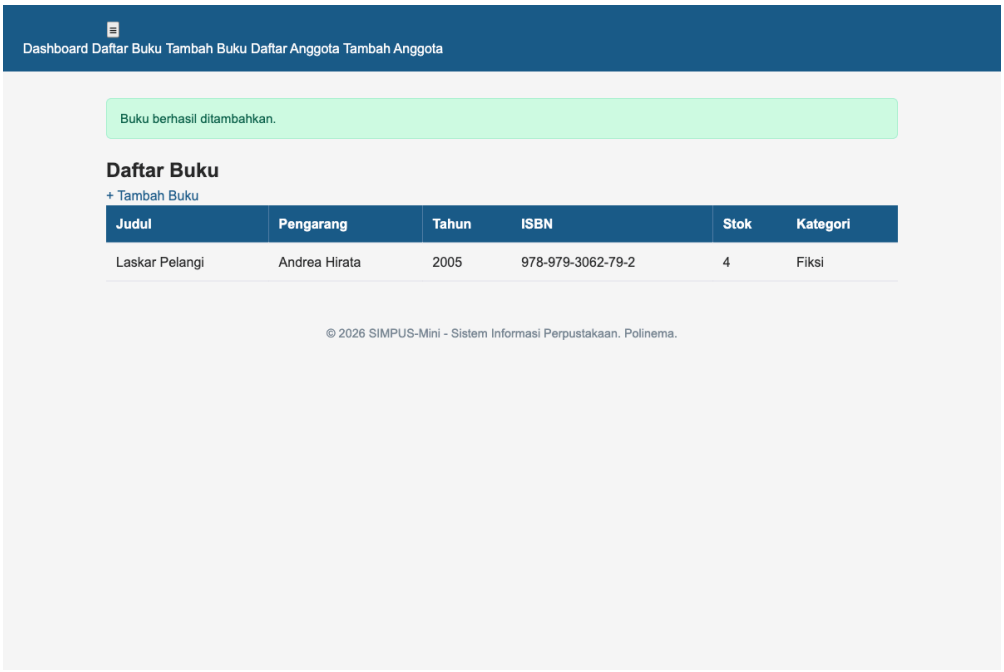
```
$_SESSION['buku'][] = [
    'judul' => $judul,
    'pengarang' => $pengarang,
    'tahun' => (int) $tahun,
    'isbn' => $isbn,
    'stok' => (int) $stok,
    'kategori' => $kategori,
];

$_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Buku berhasil ditambahkan.'];
header('Location: list.php');
exit;
```

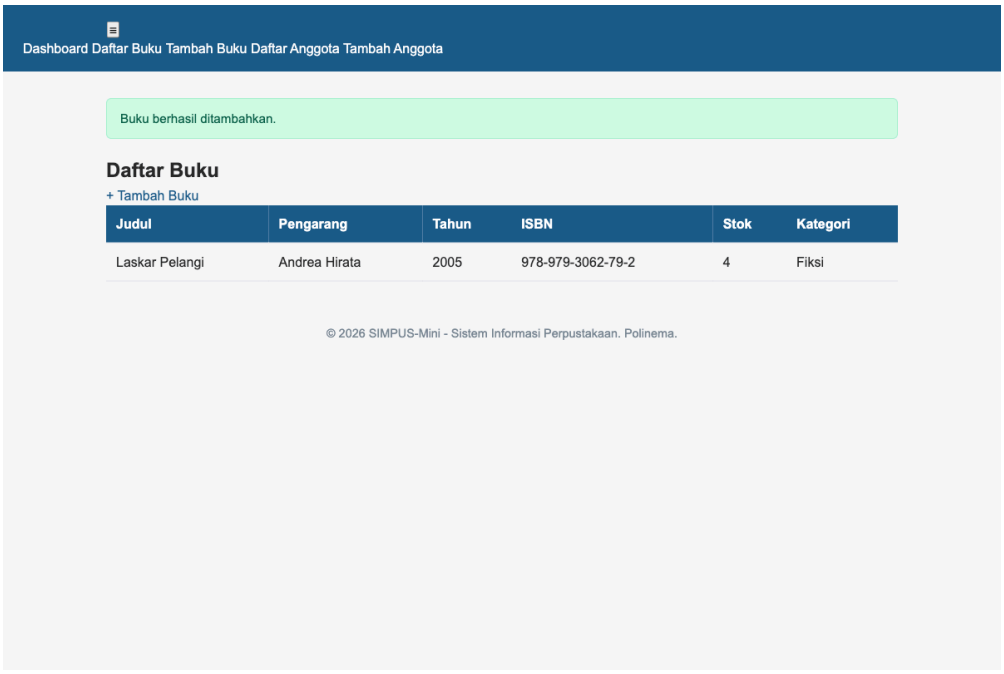
Panggilan `exit;` wajib disertakan tepat setelah `header('Location: ...')` untuk menghentikan pemrosesan skrip PHP lebih lanjut di latar belakang.

2.3.2. 3.2 Notifikasi Flash Sekali-Pakai (Flash Message)

Pada saat halaman tujuan `list.php` dibuka, komponen `includes/header.php` memeriksa keberadaan `$_SESSION['flash']`. Pesan notifikasi sukses berwarna hijau ditampilkan, dan variabel tersebut langsung dihapus dari memori server melalui perintah `unset($_SESSION['flash'])`:



Gambar 1.4: Notifikasi Flash Berhasil Ditampilkan Tepat Satu Kali Pasca-Pengalihan HTTP.



Gambar 1.5: Data Buku yang Baru Ditambahkan Berhasil Direnderdari Variabel `$_SESSION['buku']`.

2.3.3. Langkah-langkah

1. Menyimpan array asosiatif buku baru ke dalam indeks larik `$_SESSION['buku'][]`.

2. Mengirim header pengalihan `header('Location: list.php')` diikuti perintah terminasi `exit`.
3. Mengembangkan komponen perender flash message di dalam `includes/header.php` lengkap dengan pembersihan `unset()`.
4. Membuka halaman `list.php` untuk memverifikasi data baru serta memastikan flash message lenyap saat halaman di-refresh.

2.3.4. Pertanyaan

1. Apa konsekuensi fatal jika perintah `exit;` diabaikan setelah pemanggilan `header('Location: ...')`?

    - Tanpa perintah `exit;`, mesin PHP akan tetap melanjutkan eksekusi seluruh baris kode yang berada di bawahnya hingga berkas selesai, meskipun peramban telah diinstruksikan untuk berpindah halaman. Hal ini berpotensi memicu eksekusi logika yang tidak diinginkan, konsumsi sumber daya komputasi sia-sia, atau kebocoran data sensitif yang tercetak di respons stream.

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 7

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 7:

No	Pertanyaan dan Jawaban Konseptual
1	**Soal:** Jelaskan perbedaan mendasar antara eksekusi kode Server-Side (PHP) dan Client-Side (HTML/JS)! Mengapa kode sumber PHP tidak pernah terlihat saat pengguna melakukan “View Page Source” di peramban? **Jawaban:** Kode **Server-Side** seperti PHP dieksekusi secara tuntas di lingkungan komputer server web sebelum paket data respons dikirimkan melalui jaringan internet. Mesin interpreter PHP mengolah seluruh logika variabel, pengkondisian, dan manipulasi sesi, kemudian memancarkan hasil akhirnya dalam format dokumen HTML murni. Klien peramban web hanya bertindak sebagai penerima dokumen HTML tersebut. Sebaliknya, kode **Client-Side** (HTML/CSS/JS) dikirimkan dalam bentuk teks mentah dan diparsing di mesin peramban pengguna. Karena seluruh skrip PHP telah selesai diproses di server dan hanya menyisakan keluaran HTML murni, pengguna yang membuka fitur **View Page Source** tidak akan pernah melihat sebaris pun kode logika PHP aslinya.
2	**Soal:** Apa fungsi konstanta `__DIR__` saat menyertakan berkas menggunakan `require_once __DIR__ . '/../includes/header.php'`, dan mengapa teknik modularisasi ini mematuhi prinsip DRY (Don’t Repeat Yourself)? **Jawaban:** Konstanta bawaan `__DIR__` mengembalikan path direktori absolut dari sistem berkas tempat skrip yang sedang aktif berada. Penerapan `__DIR__` menghilangkan ambiguitas penelusuran path yang kerap terjadi jika hanya mengandalkan path relatif biasa saat skrip dipanggil

No	Pertanyaan dan Jawaban Konseptual
	dari working directory yang bervariasi. Modularisasi ini mematuhi prinsip DRY karena blok elemen navigasi `<header>`, pemanggilan lembar gaya CSS, dan logika inisialisasi sesi cukup didefinisikan satu kali di berkas terpusat `includes/header.php`. Setiap pembaruan menu atau modifikasi aset global di masa depan cukup dilakukan pada satu berkas tersebut tanpa perlu menyalin ulang kode ke puluhan halaman lain.
3	**Soal:** Bagaimana mekanisme `$_SESSION` menjembatani data antar-permintaan HTTP yang bersifat stateless? Mengapa data pada `$_SESSION` di Jobsheet 7 bersifat sementara (ephemeral)? **Jawaban:** Protokol transfer HTTP beroperasi tanpa status (**stateless**), yang berarti setiap permintaan dianggap independen tanpa memori terhadap request sebelumnya. PHP menjembatani keterbatasan ini melalui mekanisme Session: fungsi `session_start()` menerbitkan identifier sesi unik yang ditransmisikan ke peramban klien melalui cookie HTTP bernama `PHPSESSID`. Pada setiap request selanjutnya, peramban secara otomatis melampirkan cookie tersebut sehingga server dapat mengaitkannya kembali dengan ruang penyimpanan memori atau berkas sesi terkait di server melalui variabel superglobal `$_SESSION`. Data pada Jobsheet 7 bersifat sementara (**ephemeral**) karena disimpan di dalam media penyimpanan sementara server yang masa berlakunya terikat pada siklus hidup sesi peramban. Jika sesi kadaluarsa, peramban ditutup sepenuhnya, atau layanan web server dihentikan, data tersebut akan musnah permanen.
4	**Soal:** Mengapa validasi sisi server (Server-Side Validation) pada `proses_tambah.php` wajib diimplementasikan meskipun form input sudah memiliki atribut required dan validasi JavaScript di sisi klien? **Jawaban:** Validasi sisi klien beroperasi di lingkungan eksekusi peramban

No	Pertanyaan dan Jawaban Konseptual
	pengguna yang sepenuhnya berada di luar kendali pengembang (**untrusted execution environment**). Pengguna tingkat lanjut dapat dengan mudah mematikan fitur JavaScript melalui konfigurasi browser, mencabut atribut `required` lewat DevTools Elements Inspector, atau langsung mengirimkan payload HTTP POST berbahaya menggunakan peralatan otomasi seperti cURL dan Postman. Oleh karena itu, validasi sisi server merupakan benteng integritas data yang wajib dan mutlak karena dijalankan di lingkungan server tepercaya yang tidak dapat dimanipulasi oleh klien.
5	**Soal:** Jelaskan alur kerja pola Post/Redirect/Get (PRG) dan bagaimana mekanisme Flash Message (`$_SESSION['flash']` dan `unset()`) mencegah duplikasi data saat pengguna me-refresh halaman! **Jawaban:** Pola arsitektur PRG memisahkan tugas mutasi data dari presentasi hasil akhir: 1. **POST**: Formulir mentransmisikan data ke skrip pemroses (`proses_tambah.php`). 2. **Redirect**: Setelah data divalidasi dan disimpan, server merespons dengan header HTTP 302 `header('Location: list.php')` diikuti `exit;`. 3. **GET**: Peramban klien mengeksekusi permintaan HTTP GET baru ke URL target `list.php`. Pemisahan ini memastikan bahwa jika pengguna menekan tombol refresh (F5), peramban hanya me-refresh request GET aman dan tidak memicu pengiriman ulang formulir (**form resubmission**). Mekanisme **Flash Message** menampung pesan status pada variabel `$_SESSION['flash']`. Ketika halaman tujuan me-render pesan tersebut, eksekusi fungsi `unset($_SESSION['flash'])` langsung memusnahkannya dari memori sesi sehingga notifikasi hanya tampil tepat satu kali dan tidak muncul kembali pada kunjungan halaman berikutnya.